

Correctness Proofs: Distance Calculation and Path Reconstruction

1 Recursive Distance Calculation

Source Code

Listing 1: recursive_distance_calculation

```
1 def recursive_distance_calculation(start_node, end_node, neighbor_dict):
2     if start_node == end_node:
3         return 0
4     if neighbor_dict[end_node] == start_node:
5         return 1
6     else:
7         return 1 + recursive_distance_calculation(
8             start_node, neighbor_dict[end_node], neighbor_dict
9         )
```

Correctness of recursive_distance_calculation

Assumption. *BFS works correctly and gives us the best neighbor.*

Base Case. If start and end node are the same, there is no distance between them, return 0. If end node's neighbor is the start node, they have a distance of 1.

Induction Hypothesis. Assume that k is the distance from `neighbor_dict[end_node]` to the `start_node`, want to prove $k + 1$ is the distance to end node.

Explanation and Conclusion. We assume that the distance from any node excluding the end node to `start_node` is k . Note that we are in case 3, where our end node and start node are not identical and that our end node's immediate neighbor is not the `start_node`. In this case, we do the following:

`1 + recursive_distance(start_node, neighbor_dict[end_node], neighbor_dict).`

We add 1 and we proceed to move to the end node's immediate neighbor. By our induction hypothesis, we know that k is the distance from `neighbor_dict[end_node]` to the `start_node`. Since `neighbor_dict[end_node]` to the `start_node` is k and we added 1 for the first call:

`1 + recursive_distance(start_node, neighbor_dict[end_node], neighbor_dict),`

we prove that $k + 1$ is the distance to the end node. Note that this process is guaranteed to terminate since we applied BFS, meaning that as we follow down the neighbors of the end node, we are guaranteed to hit the start node, terminating our function.

2 Path Reconstruction (get_path)

Source Code

Listing 2: get_path

```
1 def get_path(start_node, end_node, neighbor_dict):
2     path_look_up = []
3     while True:
4         if end_node == None:
5             break
6         path_look_up.append(end_node)
7         end_node = neighbor_dict[end_node]
8     path_look_up.reverse()
9     return path_look_up
```

Correctness of get_path

Assumption. *BFS works correctly and gives us the best neighbor.*

Invariant. At any given point, `path_look_up` contains the best path from `target` to the current `end_node`.

Initialization. `path_look_up` is empty, the loop has not started, so the invariant is trivially true.

Maintenance (ignoring break). During the loop, we append `end_node`, which we know has to be part of the best path from `target` to the current `end_node`. We then assign `end_node` to its predecessor going into the next iteration. By appending `end_node` to `path_look_up` and reassigning `end_node` to its predecessor for the next iteration, we still maintain the property that `path_look_up` contains the best path from `target` to the current `end_node`.

Termination. When `end_node` is `None`, this means that we have reached the predecessor of `start_node`, since this BFS implementation returns `None` for `start_node`'s predecessor, and thus we break out of the loop. In the prior iteration, we had appended `start_node`, and thus by breaking out of the loop, we maintain our invariant.